Team Lead Software Engineer

Technical Vetting Assignment

Instructions & Requirements

Team Lead Software Engineer — Technical Vetting Assignment

Take-Home Assignment: Payments Integration Service (.NET + SQL + AWS)

Context

Zapper customers can pay a merchant by scanning their QR code in the Zapper app. The customer confirms the amount and chooses to pay using a card they previously added to their account.

The card details are not sent with each payment. When a customer adds a card, the payment provider returns a token representing that card. Zapper stores this token and uses it for future payments.

When a customer makes a payment, the Zapper app sends the request to Zapper's backend. The backend then submits the payment to Acme Pay, our payment service provider, and tracks the payment through its lifecycle.

Zapper operates as a payment facilitator. Merchants accepting payments through Zapper are registered as sub-merchants under Zapper's account. Each payment therefore needs to include information about the relevant sub-merchant so that the acquirer can process the transaction correctly and Zapper can settle the merchant afterwards. Zapper's acquiring bank is FNB.

The service you are building is responsible for storing sub-merchants and customers' stored card tokens. For this assignment, you can seed this data in your database.

A sub-merchant should include:

- Zapper's identifier for the merchant
- Trading name
- City
- Region
- Country
- Merchant Category Code (MCC)

A stored card should include:

- The token issued by Acme Pay
- The customer it belongs to
- The customer's email address

When the app requests a payment, it will identify the customer and sub-merchant. Your service is responsible for looking up the information required to construct and submit the payment request to Acme Pay.

Acme Pay's API is documented in **Appendix A**. There are a few behaviours worth keeping in mind when designing the integration: Acme can be slow to respond, notifications may be delivered more than once, and notifications are not guaranteed to arrive in order.

Our services run on AWS, so the solution should be containerised and designed with a production environment in mind.

We also expect engineers to use AI as part of their development workflow. This assignment should demonstrate how you use AI to help you work, while still showing your own engineering judgement.

Requirements

Technology

Use:

- C# / .NET 8
- A SQL database: PostgreSQL, SQL Server or SQLite
- A containerised setup that could reasonably be deployed to AWS

The solution must run locally. You do not need to deploy it to AWS.

Integration requirements

Acme Pay is fictional. Build a stub that behaves according to the API described in Appendix A and configure your service to communicate with it.

The Acme stub is supporting infrastructure for the assignment rather than something we are assessing. Feel free to generate most or all of it using AI if that saves time. We would prefer you spend your time on the service and integration itself.

Your solution should also meet the following requirements:

- Configuration should come from the environment.
- Outbound calls to Acme should behave appropriately under concurrent usage and should not cause the service to degrade.
- There should be enough logging and diagnostics to troubleshoot the integration when running locally.
- Add tests where you believe they provide useful coverage.

Scope

Focus your effort on the payment integration, persistence, notification handling and daily summary.

You only need enough synthetic data to demonstrate that the system works. A small seed script or routine containing a few sub-merchants, stored cards and payments is sufficient.

You do **not** need to implement:

- Authentication for callers of your API
- Pagination
- Batch operations
- Infrastructure as code
- Load testing

Acme's callbacks are part of the assignment. Assume that the notification endpoints exposed by your service are publicly accessible. Appendix A describes how Acme authenticates itself when calling each endpoint.

Task 1: Payment processing and notifications

Build a .NET 8 service that allows the Zapper app to submit and track payments.

The app should be able to request a payment using:

- A customer identifier
- A sub-merchant identifier
- An amount
- The app's own identifier for the payment

Your service should use the customer and sub-merchant records it holds to build the required Acme Pay request, submit the payment and keep a local record of it.

The app should also be able to query your service later to find out the current state of a payment.

Acme will send notifications as the payment moves through its lifecycle. Your service should process these notifications and keep its local payment state up to date.

Notifications may be delivered more than once and may arrive out of order. Processing the same notification more than once, or receiving multiple notifications relating to the same payment, should not result in duplicated or conflicting records.

Your implementation should demonstrate how it handles:

- A payment approved by the acquirer
- A payment declined by the acquirer
- The same payment submitted twice using the same app payment identifier
- Notifications delivered more than once
- Notifications arriving out of order
- A payment that is reversed after being approved
- A payment that is refunded after being approved
- A payment where Acme does not respond before the request times out

Document any assumptions you make in SOLUTION.md, including the Acme behaviours you chose to simulate in your stub.

Task 2: Data model and daily reporting

Design and implement the SQL schema required to support the payment flow.

The model should store:

- Payments
- Changes to a payment throughout its lifecycle
- Notification information
- Enough audit information to investigate and troubleshoot interactions with Acme

We also need a daily payment summary that Zapper can use as part of merchant settlement.

For a given date, the summary should be grouped by **sub-merchant and currency** and show:

- Number and total value of approved payments
- Number and total value of declined payments
- Number and total value of reversals
- Number and total value of refunds

You can expose the summary through an API endpoint or provide it as a SQL query that can be run directly.

In SOLUTION.md, explain the main choices you made in your data model, any important trade-offs, and how you would expect the schema to evolve as the system grows.

Task 3: AWS readiness and operability

Containerise the application and provide clear instructions for running it locally.

You do not need to deploy the service to AWS. Instead, use SOLUTION.md to describe how you would approach a production deployment.

Cover:

- How you would run the service on AWS
- How configuration and secrets would be managed
- How the service would scale
- How you would handle graceful shutdown while requests or other work are still in flight
- Which logs, metrics or other diagnostic signals you would use to understand the health of the service and troubleshoot problems

We are more interested in your reasoning than in a detailed infrastructure design.

Task 4: AI-assisted development

Use AI as you normally would while completing the assignment.

We are interested in **how you work with AI**, not just the code it produces.

Tools

In SOLUTION.md, tell us which AI tools you used and what you used them for.

Workflow

Describe how you approached the assignment with those tools. We would like to understand:

- How you gave the tools enough context to work effectively
- How you broke the assignment into smaller pieces
- What you delegated to AI
- What you chose to do yourself
- How you reviewed and iterated on AI-generated work
- What you would do differently if you approached the task again

AI artifacts

Please commit the useful artifacts produced as part of your AI workflow.

If your AI tool expects certain files to live in specific locations, leave them there. Examples include:

- CLAUDE.md
- AGENTS.md
- .cursorrules
- .claude/
- .cursor/rules/

Include any rules, commands or skills you created that you would reasonably share with the rest of an engineering team.

Put other AI working artifacts in an ai/ directory. These could include:

- Plans or specifications
- Prompts
- Task breakdowns
- Review notes

If your tool allows you to export sessions, include the sessions where you worked on the Acme client. Remove any personal information before committing them.

Validation

In SOLUTION.md, explain how you validated AI-generated work.

We are particularly interested in what you checked yourself, anything you rejected or rewrote, and how you satisfied yourself that the final implementation was correct.

Deliverables

Submit a single GitHub repository containing:

- All source code
- README.md with instructions for running the solution locally
- SOLUTION.md covering:
  - Your design and key trade-offs
  - Your AWS deployment approach
  - Your AI workflow and tools
  - How you validated AI-generated work
- Any AI tool configuration files in the locations expected by those tools
- An ai/ directory containing the remaining artifacts from your AI workflow

Also include a **5–10 minute video walkthrough** using Loom or a similar tool.

Use the walkthrough to:

- Explain the overall architecture
- Talk through the most important design decisions and trade-offs
- Show the parts of the implementation you think are worth highlighting

Time

We expect the assignment to take roughly **4 hours**. This is a guideline rather than a hard time limit. Take as much time as you need.

Appendix A — Acme Pay Payment Gateway API

Acme Pay Payment Gateway, API v2. Merchant integration guide. Last updated 14 January 2026.

Environments

Sandbox	https://sandbox.acmepay.example
Production	https://gateway.acmepay.example

Prefix the paths below with one of these hosts.

Authentication

Every request carries your API key as a bearer token:

```
Authorization: Bearer <ACME_API_KEY>
```

Sandbox keys are prefixed sk_test_. Your MerchantID is a GUID issued when your account is created, and it appears in the body of every request.

Conventions

Field names	PascalCase.
Message identifiers	MessageID is yours to generate and must be unique per message. Acme returns its own MessageID in the response rather than repeating yours.
Channel	Channel in the MessageHeader names where the message came from: App, Web or POS.
Amounts	Amount is a long in the lowest monetary unit of the currency. 1999 in ZAR is R19.99.
Currency	CurrencyCode is a three-letter ISO 4217 code. It is optional, and Acme applies ZAR when it is omitted.
Timestamps	TransactionDateTime is ISO 8601 without an offset, in South African Standard Time (UTC+02:00), to millisecond precision: 2026-03-01T13:56:36.533.
Forward compatibility	Acme adds fields to responses and notifications without notice. Your integration must accept fields it does not recognise.

Message envelope

Every request and response carries a MessageHeader alongside its payload.

{

"MessageHeader": {

"MessageDateTime": "2026-03-01T13:56:36.533",

"MessageID": "a5f1c204-9e77-4a3e-8f21-0b6d1c4e5a90",

"Channel": "App"

},

"PaymentRequest": {}

}

Transaction status

TransactionStatus is one of:

Success | Failure | Reversed | Complete | Refunded

TransactionStatus reports what the gateway did with your message, not what the bank decided. A transaction carrying TransactionStatus of Success and a ResponseDetail.Code other than 00 reached the acquirer and was declined. The outcome of the payment is in ResponseDetail.Code.

ResponseDetail

Field	Type	Max	Notes
Source	string	50	Where the result came from: Acquirer, EFT, 3DSecure, Application.
Code	string	3	The result code. 00 is approved.
Description	string	50	What the code means.
ClientMessage	string	100	Wording safe to show a cardholder.

Commonly seen codes:

Code	Description	ClientMessage
00	Approved or completed successfully	Approved.
05	Do not honour	Your bank declined this payment.
51	Insufficient funds	There was not enough money in the account.
54	Expired card	That card has expired.
91	Issuer or switch inoperative	The bank could not be reached. Please try again.
COMPL	EFT collection completed	Payment received.
PEND	EFT collection pending	We are waiting for your bank.

POST /api/payment

Submits a payment for processing.

Request

{

"MessageHeader": {

"MessageDateTime": "2026-03-01T13:56:36.533",

"MessageID": "a5f1c204-9e77-4a3e-8f21-0b6d1c4e5a90",

"Channel": "App"

},

"PaymentRequest": {

"MerchantID": "8B67BF88-BB8D-4EB5-ADFC-0F5C97C7ED67",

"TransactionID": "2e0e1b85-bf91-45a9-b038-282e00036919",

"OrderNumber": "MR43851259",

"Amount": 1999,

"CurrencyCode": "ZAR",

"PaymentService": "CardNotPresent",

"Card": {

"Token": "4f2a8c9e1b7d3560"

},

"CardOnFileScenario": "CIT",

"SubMerchantID": "ZAP0004182",

"MerchantCategoryCode": "5812",

"CardAcceptorName": "ZAP*CAFE DEL SOL CAPE TOWN WCZA",

"Email": "alice@example.com"

}

}

Field	Type	Max	Required	Notes
MerchantID	string	50	yes	The GUID issued to your account.

TransactionID	string	50	yes	Your identifier for this attempt. Acme stores the outcome against it for 24 hours and replays that outcome if the request is repeated.
OrderNumber	string	50	yes	Your reference for the order. Alphanumeric only. Its first 12 characters form the Retrieval Reference Number, which must be unique across your account and is what later messages about this payment quote.
Amount	long		yes	The amount in the lowest monetary unit.
CurrencyCode	string	3	no	ISO 4217. Acme applies ZAR when omitted.
PaymentService	enum		yes	CardNotPresent, CardNotPresentRecurring, AcmeEft, GooglePay or ApplePay.
Card	object		on card services	The stored card, on any CardNotPresent, CardNotPresentRecurring, GooglePay or ApplePay payment. Token is the tokenised card, issued when the card was added. Acme never accepts a card number on this endpoint.
CardOnFileScenario	enum	10	no	CIT when the cardholder is present and initiating, MIT when you are charging a stored card without them.

Email	string	254	no	The customer's email address.

Payment facilitators

A payment facilitator submits payments on behalf of sub-merchants, and every payment must carry the sub-merchant it belongs to. These fields are required on facilitator accounts:

Field	Type	Max	Notes
SubMerchantID	string	15	The identifier you assign to the sub-merchant.
MerchantCategoryCode	string	4	The sub-merchant's MCC.
CardAcceptorName	string	40	What the cardholder sees on their statement. Position-al — see below.

CardAcceptorName is a fixed 40-character field, space-padded, read by position:

Positions	Contents
1–23	Merchant name
24–36	City, up to 13 characters
37–38	The first two characters of the region code
39–40	Country code

On ABSA, FNB and Nedbank positions 1–23 take the form FacilitatorName*SubMerchantName, and the asterisk may appear only at position 4, 8 or 13. On Standard Bank positions 1–23 carry the sub-merchant name alone. Pad the field to a full 40 characters with spaces.

Response — 200 OK

{

"MessageHeader": {

"MessageDateTime": "2026-03-01T13:56:37.104",

"MessageID": "c8b2e551-3f44-4b0c-9d17-52ae8f6b1d33",

"Channel": "App"

},

"PaymentResponse": {

"TransactionID": "2e0e1b85-bf91-45a9-b038-282e00036919",

"TransactionDateTime": "2026-03-01T13:56:36.533",

"OrderNumber": "MR43851259",

"ReconID": "MR747891457",

"RetrievalReferenceNumber": "MR43851259",

"AuthCode": "238699",

"TransactionStatus": "Success",

"ResponseDetail": {

"Source": "Acquirer",

"Code": "00",

"Description": "Approved or completed successfully",

"ClientMessage": "Approved."

},

"AuthAmount": 1999

}

}

Acme never returns the card token. ReconID is Acme's reconciliation identifier for the transaction and appears on your settlement file. AuthAmount is what the bank actually authorised, which can be less than Amount on a partial authorisation.

Errors

HTTP	ResponseDetail.Code	Meaning
400	E01	The body could not be parsed, or a required field is missing.
401	E02	The API key is missing or not valid.
409	E09	The first 12 characters of OrderNumber are already in use. The response body carries the TransactionID they belong to.
429	E29	You have exceeded 20 requests per second. The Retry-After header gives the number of seconds to wait.
500, 502, 503, 504	E50	The gateway is degraded.

An error carries the same envelope, with the reason in ResponseDetail. On a 409 the TransactionID in the body is the transaction that already holds the Retrieval Reference Number, not the one you submitted.

{

"MessageHeader": {

"MessageDateTime": "2026-03-01T13:56:37.104",

"MessageID": "e41d7a08-52cb-4f19-b3d6-7a0c91ef2b55",

"Channel": "App"

},

"PaymentResponse": {

"TransactionID": "9c4e7f10-2a63-4d8b-91ef-5b027c88a4d1",

"OrderNumber": "MR43851259",

"TransactionStatus": "Failure",

"ResponseDetail": {

"Source": "Application",

"Code": "E09",

"Description": "Retrieval Reference Number already in use",

"ClientMessage": "We could not process that payment."

}

}

}

Timeouts

Acme answers most payment requests in under 400 ms, and an unresponsive issuer can hold a request for up to 30 seconds. Acme closes connections idle for 60 seconds.

A request that times out before a response arrives may or may not have reached the acquirer. Do not submit it again under a new TransactionID. Use Real Time Transaction Query to find out what happened.

GET /apirttq/query/Merchant/{MerchantID}

Real Time Transaction Query. Returns Acme's current record of a transaction.

Authentication. The same bearer token as the payment endpoint.

Pass either TransactionID or OrderNumber as a query parameter.

GET
/apirttq/query/Merchant/8B67BF88-BB8D-4EB5-ADFC-0F5C97C7ED67?Transaction-ID=2e0e1b85-bf91-45a9-b038-282e00036919

The response carries the same PaymentResponse body as a payment. A transaction that Acme has accepted but not yet resolved comes back with TransactionStatus of Processing and an empty AuthCode.

404 when neither identifier matches a transaction on the account.

Payment Advice notifications

Acme POSTs a JSON notification to the endpoint registered against your account whenever a transaction reaches an outcome.

Authentication. Acme presents the HTTP Basic credentials you supplied when the endpoint was registered (ACME_ADVICE_USERNAME and ACME_ADVICE_PASSWORD). Reject a delivery that does not carry them.

Acknowledgement. Your endpoint must answer 200. If Acme does not receive a 200, it resends the notification three times.

Payload

{

"RequestType": "Payment",

"MerchantID": "8B67BF88-BB8D-4EB5-ADFC-0F5C97C7ED67",

"TransactionID": "2e0e1b85-bf91-45a9-b038-282e00036919",

"TransactionDateTime": "2026-03-01T13:56:36.533",

"OrderNumber": "MR43851259",

"ReconID": "MR747891457",

"RetrievalReferenceNumber": "MR43851259",

"SystemsTraceAuditNumber": "7543",

"Amount": 1999,

"CurrencyCode": "ZAR",

"PaymentService": "CardNotPresent",

"TransactionStatus": "Success",

"ResponseSource": "Acquirer",

"ResponseCode": "00",

"ResponseDescription": "Approved",

"AuthCode": "238699",

"MaskedCardNumber": "541333******1429",

"CommProtocol": "REST"

}

Field	Type	Max	Required	Notes
RequestType	enum		yes	Authorize, Capture or Payment. Reversals and refunds arrive on the Transaction Status webhook instead.
MerchantID	string	50	yes	

TransactionID	string	50	yes	The identifier you submitted.
TransactionDateTime	datetime		no	When the transaction was processed.
OrderNumber	string	50	no	Your reference.
ReconID	string	50	no	Reconciliation identifier.
RetrievalReferenceNumber	string	12	yes	
SystemsTraceAuditNumber	string	6	yes	
Amount	long		yes	
CurrencyCode	string	3	no	
PaymentService	enum		yes	
TransactionStatus	enum		yes	Success, Failure or Other.
ResponseSource	string	50	yes	
ResponseCode	string	3	yes	00 is approved.
ResponseDescription	string	50	no	
AuthCode	string	6	no	The acquirer's authorisation code.
MaskedCardNumber	string	19	no	
CommProtocol	string	4	yes	REST for notifications.

Transaction Status webhook

Reversals and refunds are published by Acme Switch, a separate service, on its own endpoint. Register it in the merchant portal under **Webhooks**.

Signature. Each delivery carries an x-signature header: an HMAC-SHA256 of the exact bytes of the request body, keyed with your webhook secret (ACME_WEBHOOK_SECRET) and encoded as Base64.

Worked example. With the secret `k7Qf2mZr9XpL4vNcA8sTgH3wYbE6dJ5u` and this body on a single line:

```
{"transactionUUID":"7f3a9c12-4b8e-4d21-9f0a-6c5e2d81b4a7","transactionType":"Reversal","transactionDateTime":"2026-03-04T08:02:11.417","rrn":"MR43851259","transactionAmount":1999,"currencyCode":"ZAR","txResultCode":"00","isApproved":true}
```

The header is:

```
x-signature: KURAJlPrV3BWvWzsGBgr0EzClUSfu3itG62dM5Lqy/Y=
```

To produce a signature yourself:

```
printf '%s' "$BODY" | openssl dgst -sha256 -hmac "$ACME_WEBHOOK_SECRET" -binary | base64
```

Acknowledgement. Answer 200. Acme Switch resends three times otherwise.

Payload

{

"transactionUUID": "7f3a9c12-4b8e-4d21-9f0a-6c5e2d81b4a7",

"transactionType": "Reversal",

"transactionDateTime": "2026-03-04T08:02:11.417",

"rrn": "MR43851259",

"transactionAmount": 1999,

"currencyCode": "ZAR",

"txResultCode": "00",

"isApproved": true

}

Field	Type	Notes
transactionUUID	string	Acme Switch's identifier for this event.
transactionType	string	Reversal or Refund.
transactionDateTime	string	When the switch processed it.
rrn	string	The Retrieval Reference Number of the original payment.
transactionAmount	number	The amount in the lowest monetary unit.
currencyCode	string	ISO 4217.

txResultCode	string	00 when the switch applied the event.
isApproved	boolean	Whether the switch applied it.

A reversal or a refund can arrive days after the payment was approved. transactionAmount is the amount of this event, which on a partial refund is less than the payment's.